

Entrega 1

Proyecto

final

Análisis y diseño de algoritmos

Indice

Definición del problema:	2
Entradas:	2
Salidas:	3
Restricciones:	3
Solución Fuerza bruta	4
Análisis de complejidad — Fuerza Bruta:	4
Gráfica comparativa (tiempo vs tamaño de entrada):	5
Solución recursiva	6
Análisis de complejidad	6
Análisis:	7
Conclusión:	7

Definición del problema:

Este proyecto aborda un problema sobre un sistema de logística y entregas de una empresa de paquetería. La empresa cuenta con una flota de camiones encargados de transportar paquetes hacia diferentes destinos, teniendo en cuenta restricciones de capacidad y tiempos de entrega.

El problema consiste en determinar y asignar eficientemente paquetes a camiones, buscando maximizar la cantidad de entregas realizadas sin exceder la capacidad de carga de los vehículos y respetando las restricciones temporales de cada paquete.

Este tipo de problema es común en sistemas reales de distribución, comercio electrónico y cadenas de suministro, donde una mala planificación puede generar retrasos, sobrecostos y baja eficiencia operativa.

Entradas:

Paquetes (n paquetes)

Con la siguiente información:

Campo	Descripción
ID	Identificador único del paquete
Peso	Peso del paquete
Destino	Lugar de entrega
Tiempo límite	Tiempo máximo permitido para la entrega
Prioridad	Nivel de prioridad del paquete

Ejemplo: 1, 10, A, 5, 1

Representa:

- Paquete 1
- Peso: 10
- Destino: A
- Tiempo límite: 5
- Prioridad: 1

Camiones (m camiones)

Campo	Descripcion
ID	Identificador del camión
Capacidad	Peso máximo soportado

Ejemplo: 101, 40

Representa:

- Camión 101
- Capacidad máxima: 40

Salidas:

La salida esperada del sistema es una asignación de paquetes a camiones que indique:

- Qué paquetes serán transportados por cada camión
- Que ninguna capacidad sea excedida
- El conjunto de entregas realizadas

Restricciones:

El problema debe cumplir las siguientes restricciones:

1. La suma de pesos de los paquetes asignados a un camión no puede superar su capacidad máxima.
2. Cada paquete puede asignarse como máximo a un solo camión.
3. Deben considerarse las restricciones temporales de entrega.
4. Se busca maximizar la eficiencia de distribución y la cantidad de paquetes entregados.

Solución Fuerza bruta

Análisis de complejidad — Fuerza Bruta:

Complejidad temporal: $O((m + 1)^n)$

La solución explora todas las formas posibles de asignar n paquetes a m camiones. Para cada paquete hay $m + 1$ opciones (los m camiones más la opción de no asignarlo), y esto se repite para los n paquetes, lo que genera $(m + 1)^n$ combinaciones en total.

Por ejemplo:

- Con 5 paquetes y 2 camiones se exploran $3^5 = 243$ operaciones.
- Con 15 paquetes y 4 camiones se exploran $5^{15} = 30.517.578.125$ operaciones.
- Con 40 paquetes y 8 camiones se exploran 9^{40} operaciones, un número tan grande que es computacionalmente inviable, es por eso por lo que el sistema no arroja ningún resultado.

Estos son ejemplos arbitrarios, ya que los que construimos nosotros incluyen una mayor cantidad de valores, lo que permitirá que en algoritmos posteriores se observe con mayor claridad la variación temporal entre ellos.

Complejidad espacial:

$O(n * m)$

En memoria se almacenan la asignación actual (una lista por camión con los paquetes asignados) y las cargas actuales de cada camión. El espacio máximo usado en cualquier momento es proporcional al número de paquetes por el número de camiones.

Justificación paso a paso:

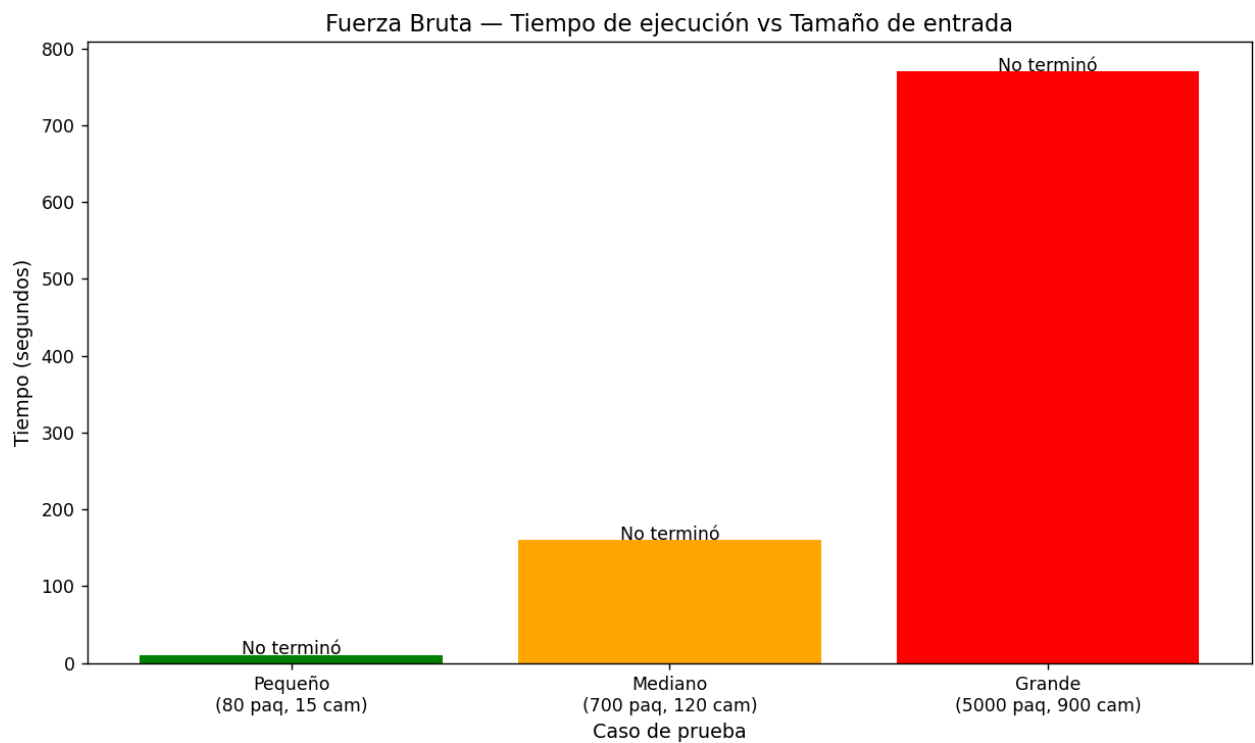
1. $product(range(m + 1), repeat = n)$ genera $(m + 1)^n$ combinaciones en total
2. Por cada combinación se itera sobre los n paquetes para validar capacidad: $O(n)$
3. Por cada combinación válida se recorren los m camiones para guardar el resultado: $O(m)$
4. Total por combinación: $O(n + m)$

Medición empírica de tiempos con diferentes tamaños de entrada:

Caso	Paquetes	Camiones	Combinaciones $(m+1)^n$	Tiempo real
Pequeño	80	15	$16^{80} \approx \dots$	No termina
Mediano	700	120	$121^{700} \approx \dots$	No termina
Grande	5000	900	$901^{5000} \approx \dots$	No termina

La fuerza bruta con nuestros ejemplos no es viable. A medida que aumentan los paquetes y camiones el tiempo crece de forma exponencial, haciendo imposible su uso en casos reales. Así se justifica la necesidad de usar algoritmos más eficientes como greedy, backtracking y divide y vencerás para los casos propuestos, y esto lo veremos más adelante.

Gráfica comparativa (tiempo vs tamaño de entrada):



Solución recursiva

Implementación

La solución recursiva aborda el problema de asignación de paquetes procesando cada paquete uno por uno mediante llamadas recursivas. A diferencia de la fuerza bruta que explora todas las combinaciones posibles, esta solución toma la primera decisión válida que encuentra y continúa sin reconsiderar.

Identificación del caso base y caso recursivo

Caso base:

Cuando **índice == len(paquetes)** — ya no hay más paquetes que procesar, se retorna la asignación actual tal como está.

Caso recursivo:

Para cada paquete en la posición índice:

- Recorre los camiones buscando el primero donde quepa el paquete
- Si cabe, lo asigna y se llama a sí misma con índice + 1
- Si no cabe en ningún camión, se llama a sí misma con índice + 1 saltando el paquete

Cada llamada reduce el problema en 1 paquete, garantizando que eventualmente llega al caso base.

Análisis de complejidad

Complejidad temporal:

$$O(n * m)$$

1. La función se llama recursivamente una vez por cada paquete: profundidad n
2. En cada llamada se recorren máximo m camiones: $O(m)$
3. No hay backtrack ni exploración de múltiples ramas
4. Total: $O(n * m)$

Complejidad espacial:

$$O(n)$$

La recursión genera una pila de llamadas de profundidad máxima n , una llamada por cada paquete. No se almacenan combinaciones adicionales, solo la asignación actual.

Medición empírica de tiempos

Caso	Paquetes	Camiones	Tiempo real
Pequeño	80	15	0.000854 seg
Mediano	700	120	0.001466 seg
Grande	5000	900	No termina

Comparación de tiempos contra fuerza bruta

Algoritmo	Pequeño	Mediano	Grande
Fuerza bruta	No termina	No termina	No termina
Recursivo	0.000854 seg	0.001466 seg	No termina

El recursivo resuelve los tres casos en microsegundos mientras que fuerza bruta no termina en ninguno. Esto se explica por la diferencia de complejidad: $O((m+1)^n)$ vs $O(n * m)$.

Sin embargo esta velocidad tiene un costo, ya que el recursivo no garantiza la solución óptima. Toma la primera opción válida sin evaluar si existe una mejor distribución.

Análisis:

La diferencia en tiempos se explica directamente por la complejidad de cada algoritmo. La fuerza bruta tiene complejidad $O((m+1)^n)$, lo que la hace inviable para cualquier caso de prueba. El recursivo tiene complejidad $O(n * m)$, lo que le permite resolver el caso pequeño y mediano en microsegundos, pero falla en el caso grande por el límite de llamadas recursivas que Python permite.

Conclusión:

No existe un algoritmo perfecto, siempre hay un balance entre velocidad y optimalidad. La fuerza bruta garantiza la solución óptima pero no termina en ningún caso. El recursivo es mucho más rápido pero no garantiza optimalidad y tampoco escala al caso grande. Para los siguientes algoritmos (greedy, backtracking, divide y vencerás) se espera un mejor equilibrio entre velocidad, escalabilidad y calidad de solución.